Seoul Rental Bike Sharing Demand

Screenshots

Submitted by - Apeetha S

Batch - DSB02

Importing libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

✓ 3.6s

Loading Dataset

```
#load dataset
df=pd.read_csv('SeoulBikeData.csv', encoding="latin1")
```

Initial EDA

Top 5 rows

```
df.head()
```

✓ 0.0s Python

	Date	Rented Bike Count	Hour	Temperature(°C)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Dew point temperature(°C)	Solar Radiation (MJ/m2)	Rainfall(mm)	Snowfall (cm)	Seasons	Holiday	Functioning Day
0	01/12/2017	254	0	-5.2	37	2.2	2000	-17.6	0.0	0.0	0.0	Winter	No Holiday	Yes
1	01/12/2017	204	1	-5.5	38	0.8	2000	-17.6	0.0	0.0	0.0	Winter	No Holiday	Yes
2	01/12/2017	173	2	-6.0	39	1.0	2000	-17.7	0.0	0.0	0.0	Winter	No Holiday	Yes
3	01/12/2017	107	3	-6.2	40	0.9	2000	-17.6	0.0	0.0	0.0	Winter	No Holiday	Yes
4	01/12/2017	78	4	-6.0	36	2.3	2000	-18.6	0.0	0.0	0.0	Winter	No Holiday	Yes

Last 5 rows

```
df.tail()
```

✓ 0.0s Python

	Date	Rented Bike Count	Hour	Temperature(°C)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Dew point temperature(°C)	Solar Radiation (MJ/m2)	Rainfall(mm)	Snowfall (cm)	Seasons	Holiday	Functioning Day
8755	30/11/2018	1003	19	4.2	34	2.6	1894	-10.3	0.0	0.0	0.0	Autumn	No Holiday	Yes
8756	30/11/2018	764	20	3.4	37	2.3	2000	-9.9	0.0	0.0	0.0	Autumn	No Holiday	Yes
8757	30/11/2018	694	21	2.6	39	0.3	1968	-9.9	0.0	0.0	0.0	Autumn	No Holiday	Yes
8758	30/11/2018	712	22	2.1	41	1.0	1859	-9.8	0.0	0.0	0.0	Autumn	No Holiday	Yes
8759	30/11/2018	584	23	1.9	43	1.3	1909	-9.3	0.0	0.0	0.0	Autumn	No Holiday	Yes

Dataset Shape

```
df.shape
```

```
(8760, 14)
```

Dataset info

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 8760 entries, 0 to 8759
Data columns (total 14 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Date                       8760 non-null   str
 1   Rented Bike Count          8760 non-null   int64
 2   Hour                       8760 non-null   int64
 3   Temperature(°C)            8760 non-null   float64
 4   Humidity(%)                8760 non-null   int64
 5   Wind speed (m/s)           8760 non-null   float64
 6   Visibility (10m)           8760 non-null   int64
 7   Dew point temperature(°C)  8760 non-null   float64
 8   Solar Radiation (MJ/m2)    8760 non-null   float64
 9   Rainfall(mm)               8760 non-null   float64
 10  Snowfall (cm)              8760 non-null   float64
 11  Seasons                    8760 non-null   str
 12  Holiday                    8760 non-null   str
 13  Functioning Day            8760 non-null   str
dtypes: float64(6), int64(4), str(4)
memory usage: 958.3 KB
```

Dataset Statistical summary

```
df.describe()
```
✓ 0.3s

	Rented Bike Count	Hour	Temperature(°C)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Dew point temperature(°C)	Solar Radiation (MJ/m2)	Rainfall(mm)	Snowfall (cm)
count	8760.000000	8760.000000	8760.000000	8760.000000	8760.000000	8760.000000	8760.000000	8760.000000	8760.000000	8760.000000
mean	704.602055	11.500000	12.882922	58.226256	1.724909	1436.825799	4.073813	0.569111	0.148687	0.075068
std	644.997468	6.922582	11.944825	20.362413	1.036300	608.298712	13.060369	0.868746	1.128193	0.436746
min	0.000000	0.000000	-17.800000	0.000000	0.000000	27.000000	-30.600000	0.000000	0.000000	0.000000
25%	191.000000	5.750000	3.500000	42.000000	0.900000	940.000000	-4.700000	0.000000	0.000000	0.000000
50%	504.500000	11.500000	13.700000	57.000000	1.500000	1698.000000	5.100000	0.010000	0.000000	0.000000
75%	1065.250000	17.250000	22.500000	74.000000	2.300000	2000.000000	14.800000	0.930000	0.000000	0.000000
max	3556.000000	23.000000	39.400000	98.000000	7.400000	2000.000000	27.200000	3.520000	35.000000	8.800000

```
df.columns.tolist()
```
✓ 0.0s

```
['Date',
 'Rented Bike Count',
 'Hour',
 'Temperature(°C)',
 'Humidity(%)',
 'Wind speed (m/s)',
 'Visibility (10m)',
 'Dew point temperature(°C)',
 'Solar Radiation (MJ/m2)',
 'Rainfall(mm)',
 'Snowfall (cm)',
 'Seasons',
 'Holiday',
 'Functioning Day']
```

Columns of the dataset

checking for missing values

```
df.isnull().sum()
```

```
Date                          0
Rented Bike Count             0
Hour                          0
Temperature(°C)               0
Humidity(%)                   0
Wind speed (m/s)              0
Visibility (10m)              0
Dew point temperature(°C)     0
Solar Radiation (MJ/m2)       0
Rainfall(mm)                  0
Snowfall (cm)                 0
Seasons                       0
Holiday                       0
Functioning Day               0
dtype: int64
```

Checking for Duplicates

```
df.dtypes
```

✓ 0.0s

```
df.duplicated().sum()
```

2]

```
np.int64(0)
```

Datatypes in the dataset

```
Visibility (10m)              int64
Dew point temperature(°C)     float64
Solar Radiation (MJ/m2)       float64
Rainfall(mm)                  float64
Snowfall (cm)                 float64
Seasons                       str
Holiday                       str
Functioning Day               str
dtype: object
```

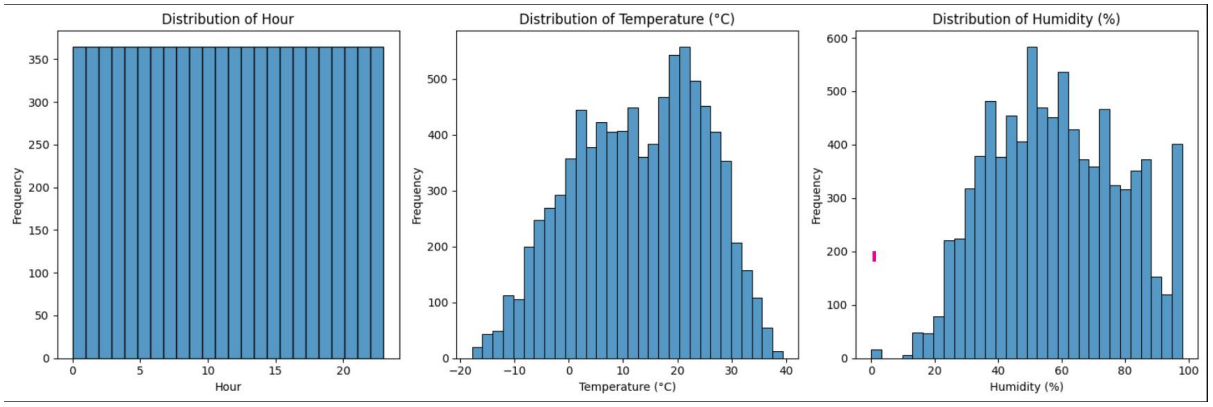
Conversion of date column – datetime

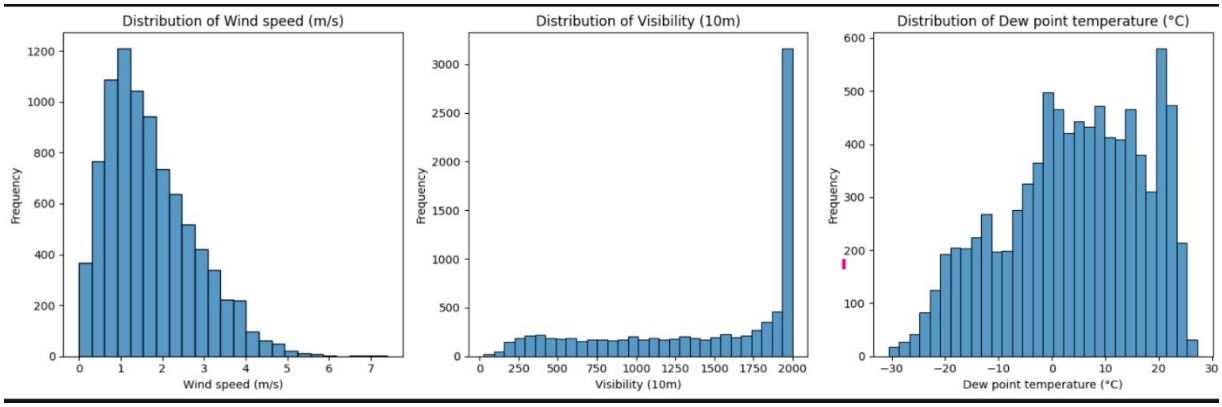
```
df['Date'] = pd.to_datetime(df['Date'],format='%d/%m/%Y')
df[['Date']].head()
```

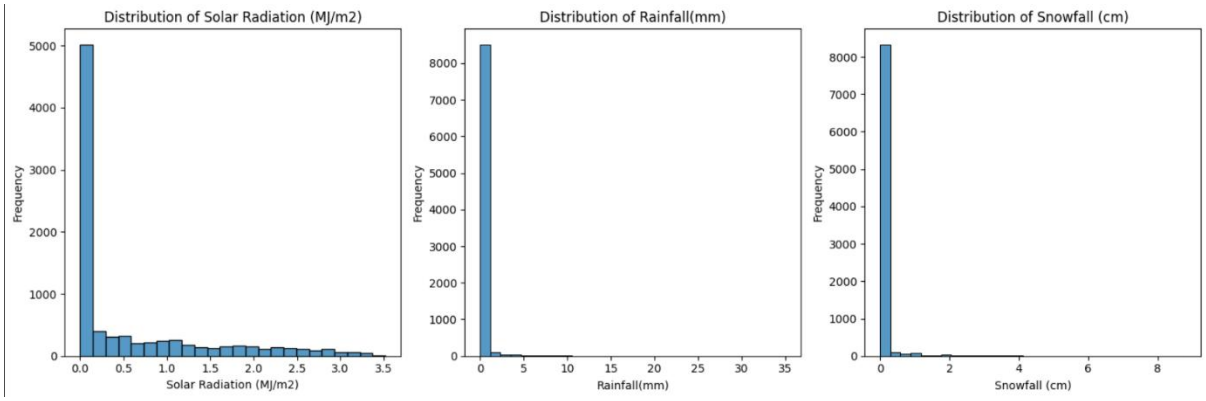
	Date
0	2017-12-01
1	2017-12-01
2	2017-12-01
3	2017-12-01
4	2017-12-01

Univariate Analysis

```
#Univariate analysis of Hour, Temperature and Humidity columns.
#using subplots for easier comparision and space optimization.

fig, axes = plt.subplots(1, 3, figsize=(15,5))

sns.histplot(df['Hour'], bins=24, ax=axes[0])
axes[0].set_title('Distribution of Hour')
axes[0].set_xlabel('Hour')
axes[0].set_ylabel('Frequency')

sns.histplot(df['Temperature(°C)'], bins=30, ax=axes[1])
axes[1].set_title('Distribution of Temperature (°C)')
axes[1].set_xlabel('Temperature (°C)')
axes[1].set_ylabel('Frequency')

sns.histplot(df['Humidity(%)'], bins=30, ax=axes[2])
axes[2].set_title('Distribution of Humidity (%)')
axes[2].set_xlabel('Humidity (%)')
axes[2].set_ylabel('Frequency')

plt.tight_layout()
plt.show()
```


Distribution of Hour
Frequency
Hour
Distribution of Temperature (°C)
Frequency
Temperature (°C)
Distribution of Humidity (%)
Frequency
Humidity (%)


Distribution of Wind speed (m/s)
Frequency
Wind speed (m/s)
Distribution of Visibility (10m)
Frequency
Visibility (10m)
Distribution of Dew point temperature (°C)
Frequency
Dew point temperature (°C)


Distribution of Solar Radiation (MJ/m2)
Frequency
Solar Radiation (MJ/m2)
Distribution of Rainfall(mm)
Frequency
Rainfall(mm)
Distribution of Snowfall (cm)
Frequency
Snowfall (cm)

Skewness Analysis

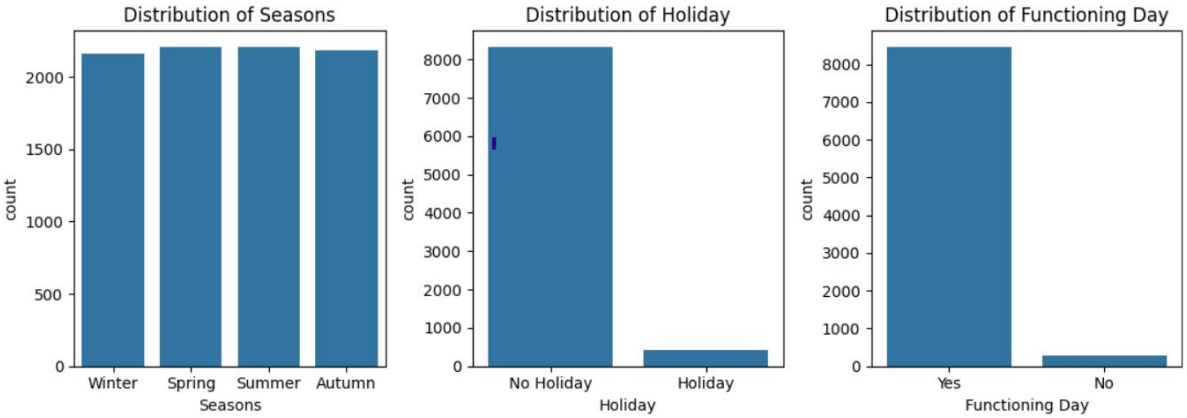
```
# Calculate skewness of numerical features
skewness = numerical_cols.skew().sort_values(ascending=False)

skewness_df = pd.DataFrame({
    'Feature': skewness.index,
    'Skewness': skewness.values
})

skewness_df
```

✓ 0.0s

	Feature	Skewness
0	Rainfall(mm)	14.533232
1	Snowfall (cm)	8.440801
2	Solar Radiation (MJ/m2)	1.504040
3	Rented Bike Count	1.153428
4	Wind speed (m/s)	0.890955
5	Humidity(%)	0.059579
6	Hour	0.000000
7	Temperature(°C)	-0.198326
8	Dew point temperature(°C)	-0.367298
9	Visibility (10m)	-0.701786

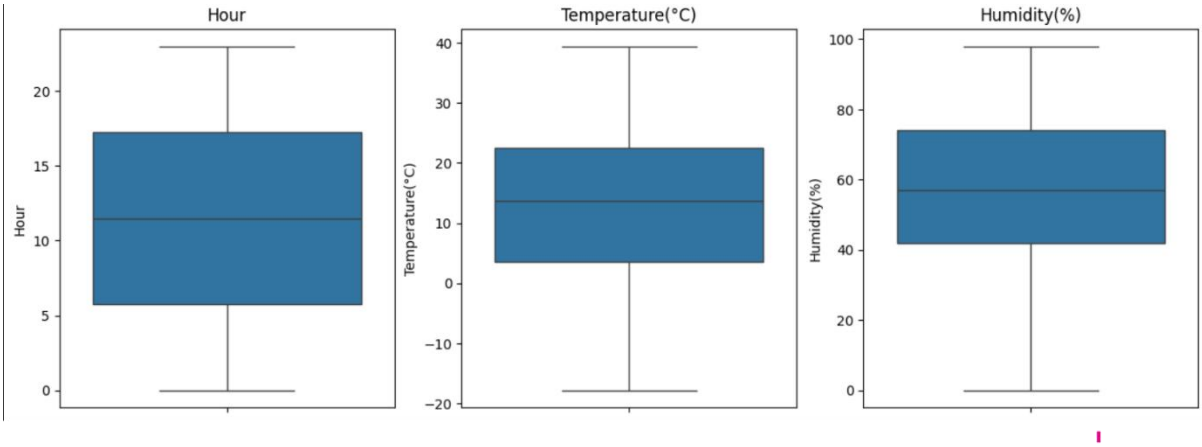
Categorical Columns – countplots

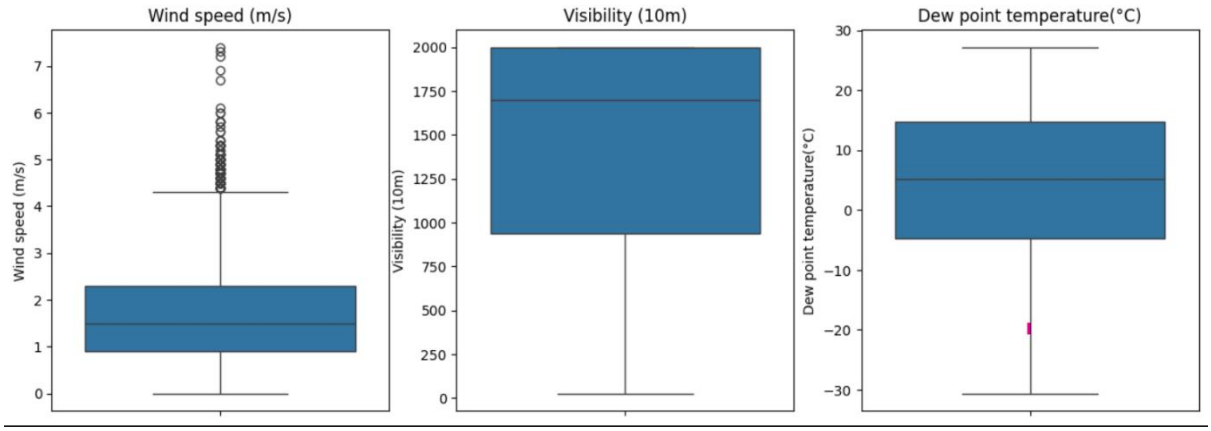
```
#Univariate analysis of categorical variables(count plots).
#using subplots for easier comparision and space optimization.

fig, axes = plt.subplots(1, 3, figsize=(11,4))

sns.countplot(data=df, x='Seasons', ax=axes[0])
axes[0].set_title('Distribution of Seasons')

sns.countplot(data=df, x='Holiday', ax=axes[1])
axes[1].set_title('Distribution of Holiday')

sns.countplot(data=df, x='Functioning Day', ax=axes[2])
axes[2].set_title('Distribution of Functioning Day')

plt.tight_layout()
plt.show()
```

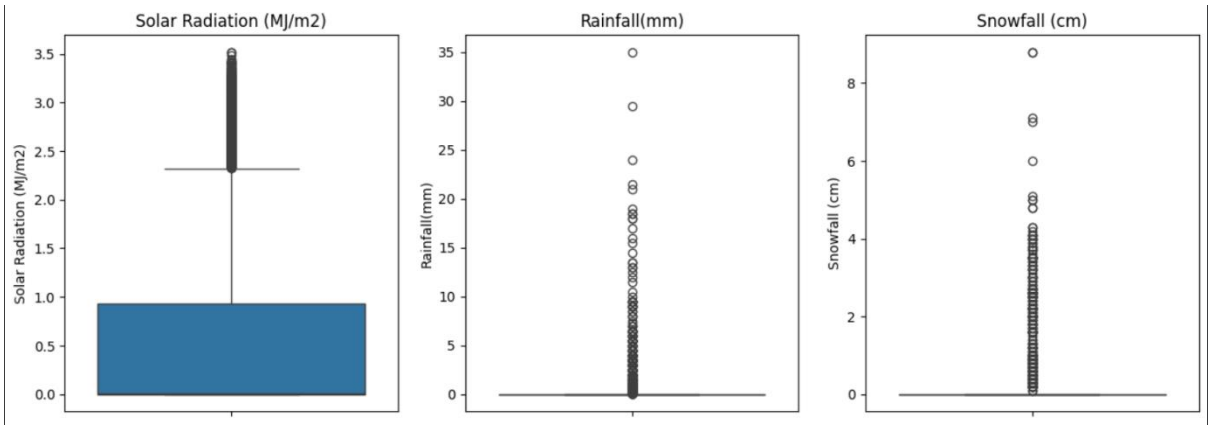


Boxplots

```
#box plots for Hour,Temperature(°C),Humidity(%) columns
fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(15,5))

sns.boxplot(y=df['Hour'],ax=axes[0])
axes[0].set_title('Hour')

sns.boxplot(y=df['Temperature(°C)'],ax=axes[1])
axes[1].set_title('Temperature(°C)')

sns.boxplot(df['Humidity(%)'],ax=axes[2])
axes[2].set_title('Humidity(%)')

plt.show()
```

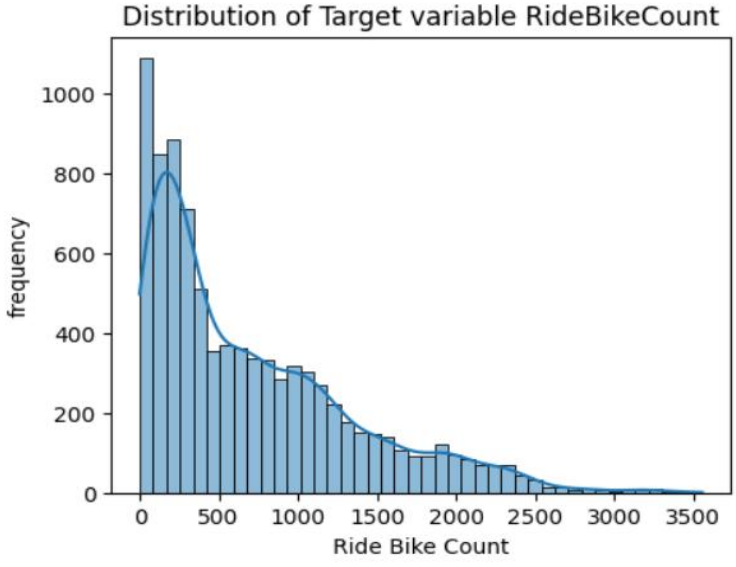




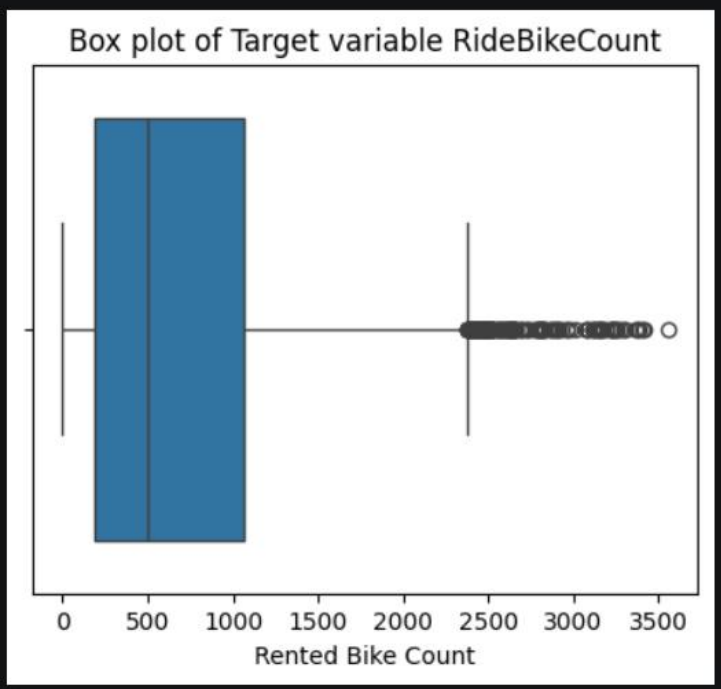



```
plt.figure(figsize=(5,4))
sns.histplot(df['Rented Bike Count'], kde=True)
plt.title("Distribution of Target variable RideBikeCount")
plt.xlabel("Ride Bike Count")
plt.ylabel("frequency")
plt.show()
```

Target Variable Analysis

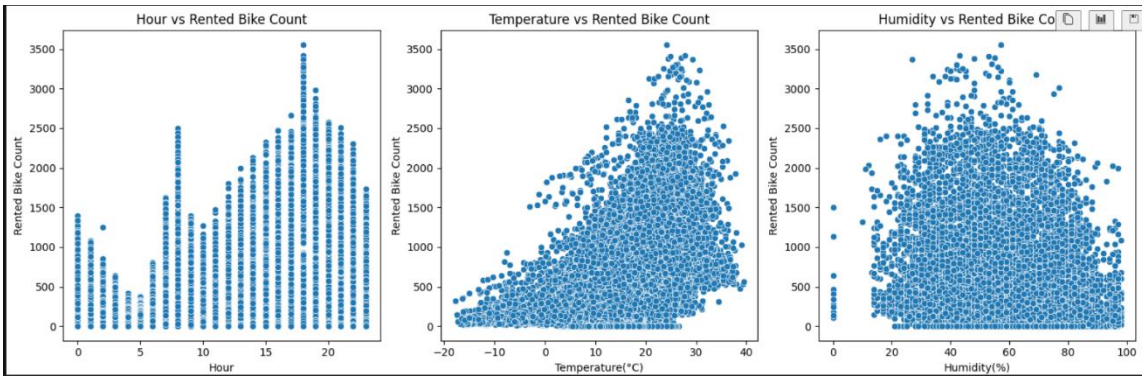


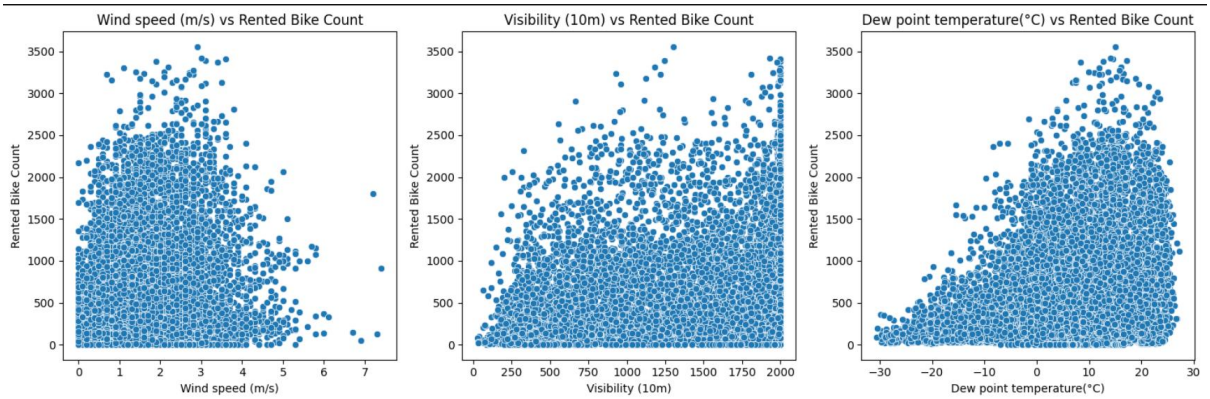
```
plt.figure(figsize=(5,4))
sns.boxplot(x=df['Rented Bike Count'])
plt.title("Box plot of Target variable RideBikeCount")
plt.show()
```

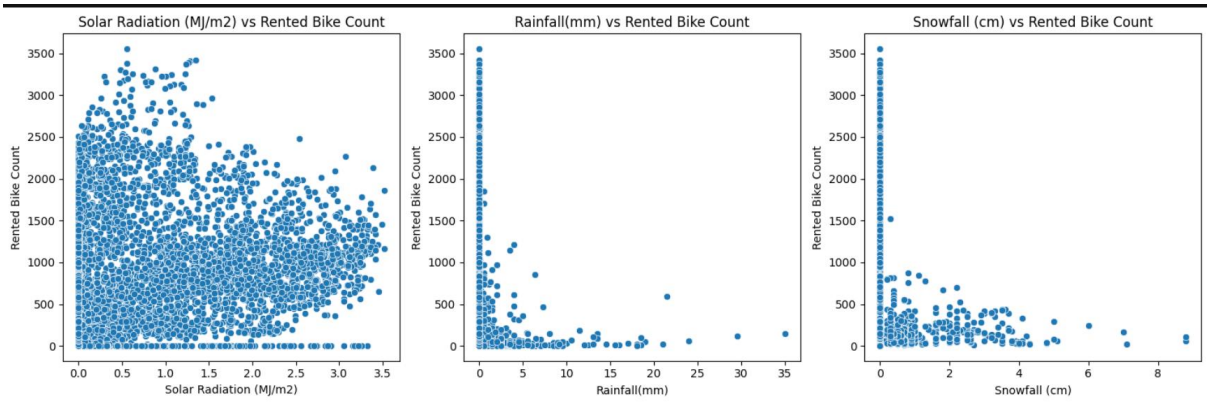


Bivariate Analysis

```
#Scatter Plots

fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(15,5))

sns.scatterplot(data=df,x='Hour',y='Rented Bike Count',ax=axes[0])
axes[0].set_title('Hour vs Rented Bike Count')

sns.scatterplot(data=df,x='Temperature(°C)',y='Rented Bike Count',ax=axes[1])
axes[1].set_title('Temperature vs Rented Bike Count')

sns.scatterplot(data=df,x='Humidity(%)',y='Rented Bike Count',ax=axes[2])
axes[2].set_title('Humidity vs Rented Bike Count')

plt.tight_layout()
plt.show()
```

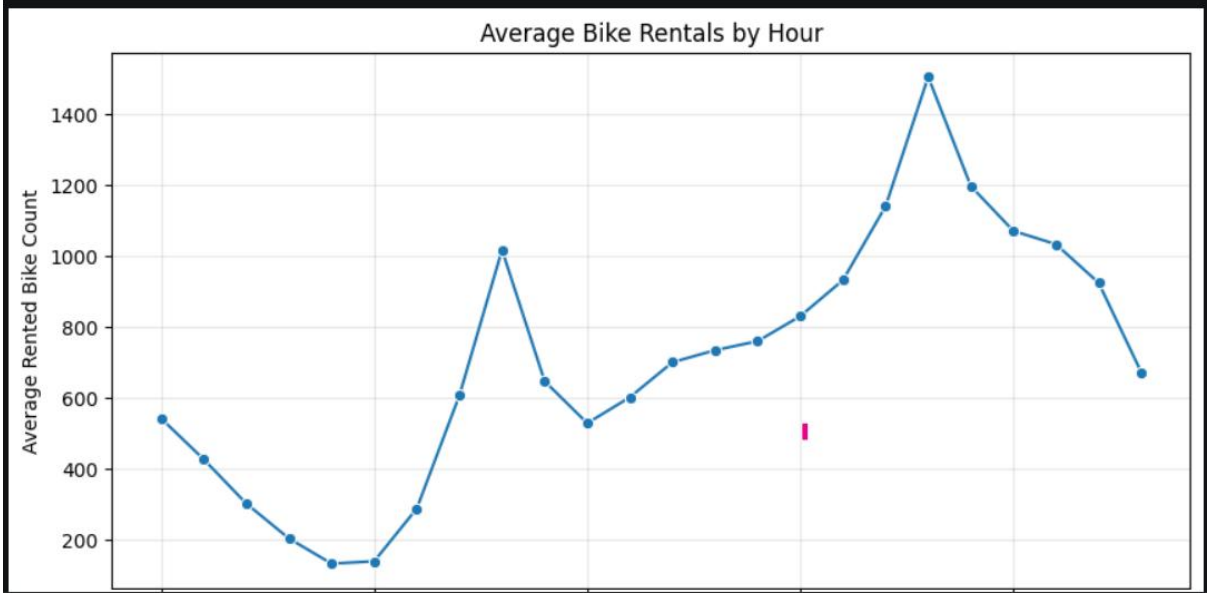






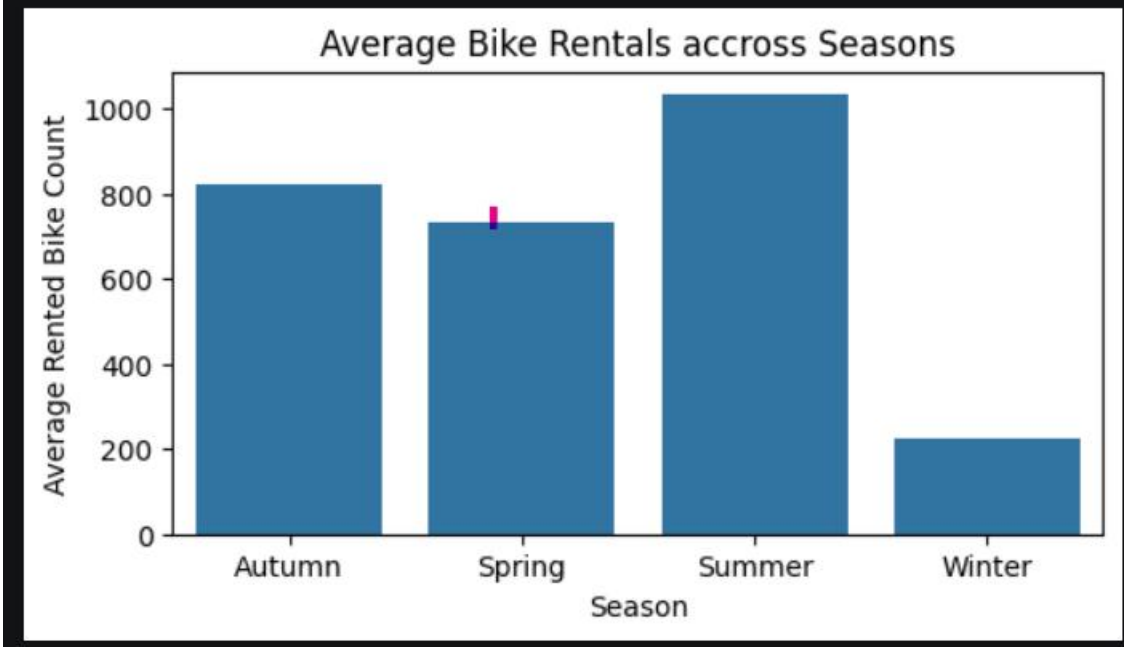
Average Ride count per hour

```
plt.figure(figsize=(10,5))
sns.lineplot(data=Hourly_Bike_count,
             x='Hour',
             y='Rented Bike Count',
             marker='o')

plt.title('Average Bike Rentals by Hour')
plt.xlabel('Hour')
plt.ylabel('Average Rented Bike Count')
plt.grid(alpha=0.3)
plt.show()
```



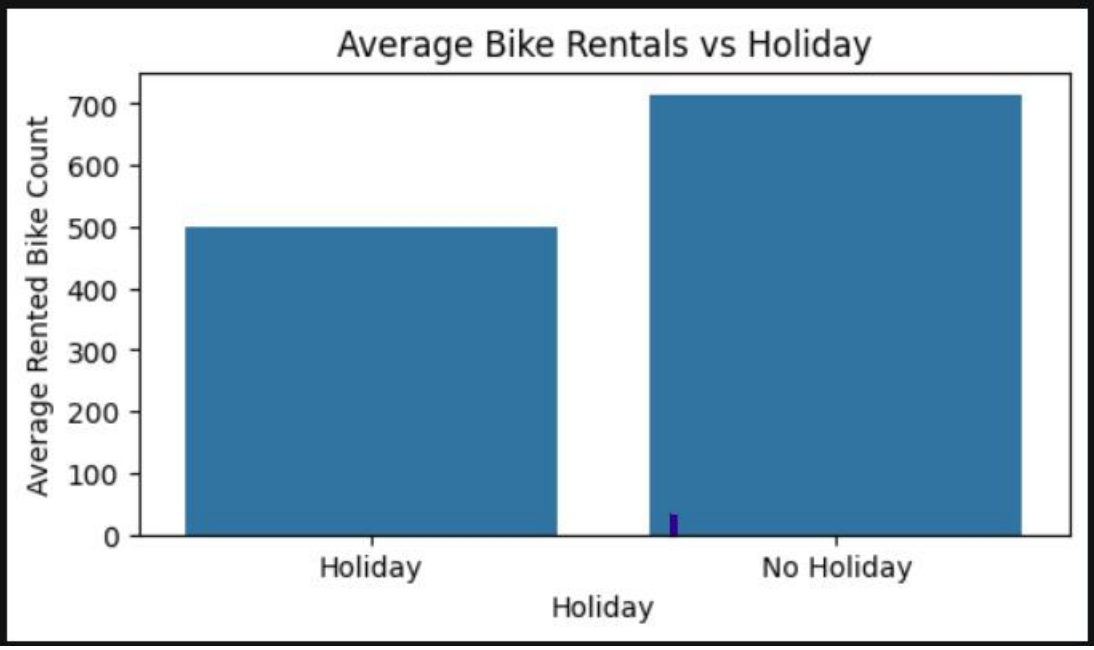
Bike Rentals Across Seasons

```
plt.figure(figsize=(6,3))
sns.barplot(data=season_Bike_count,
            x='Seasons',
            y='Rented Bike Count')

plt.title('Average Bike Rentals accross Seasons')
plt.xlabel('Season')
plt.ylabel('Average Rented Bike Count')

plt.show()
```

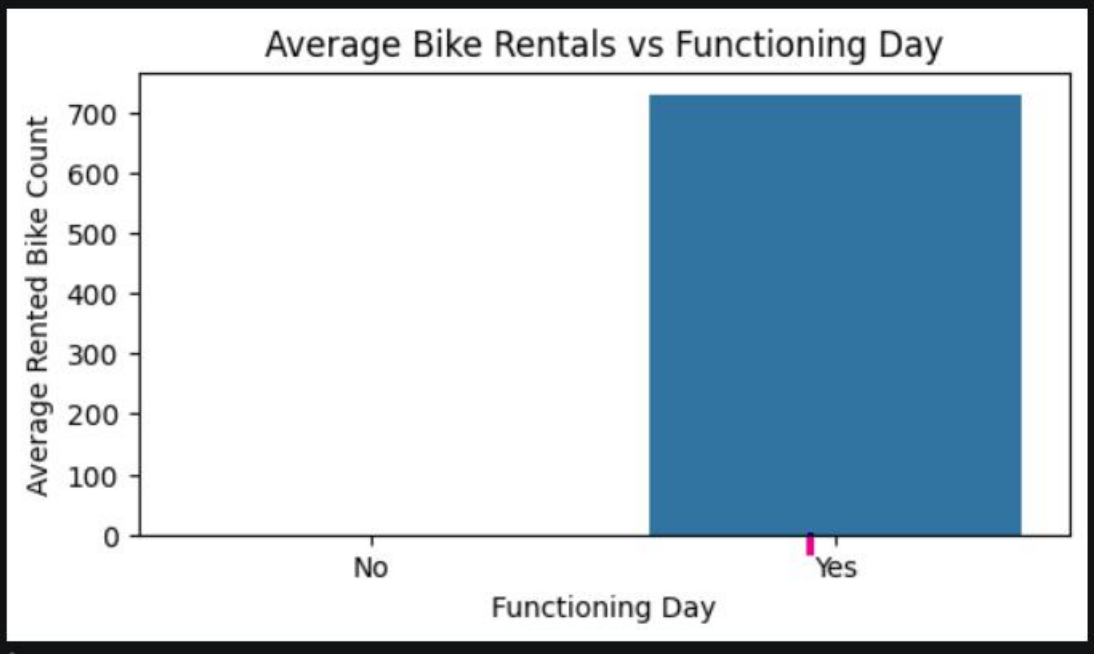


Holidays vs No Holiday

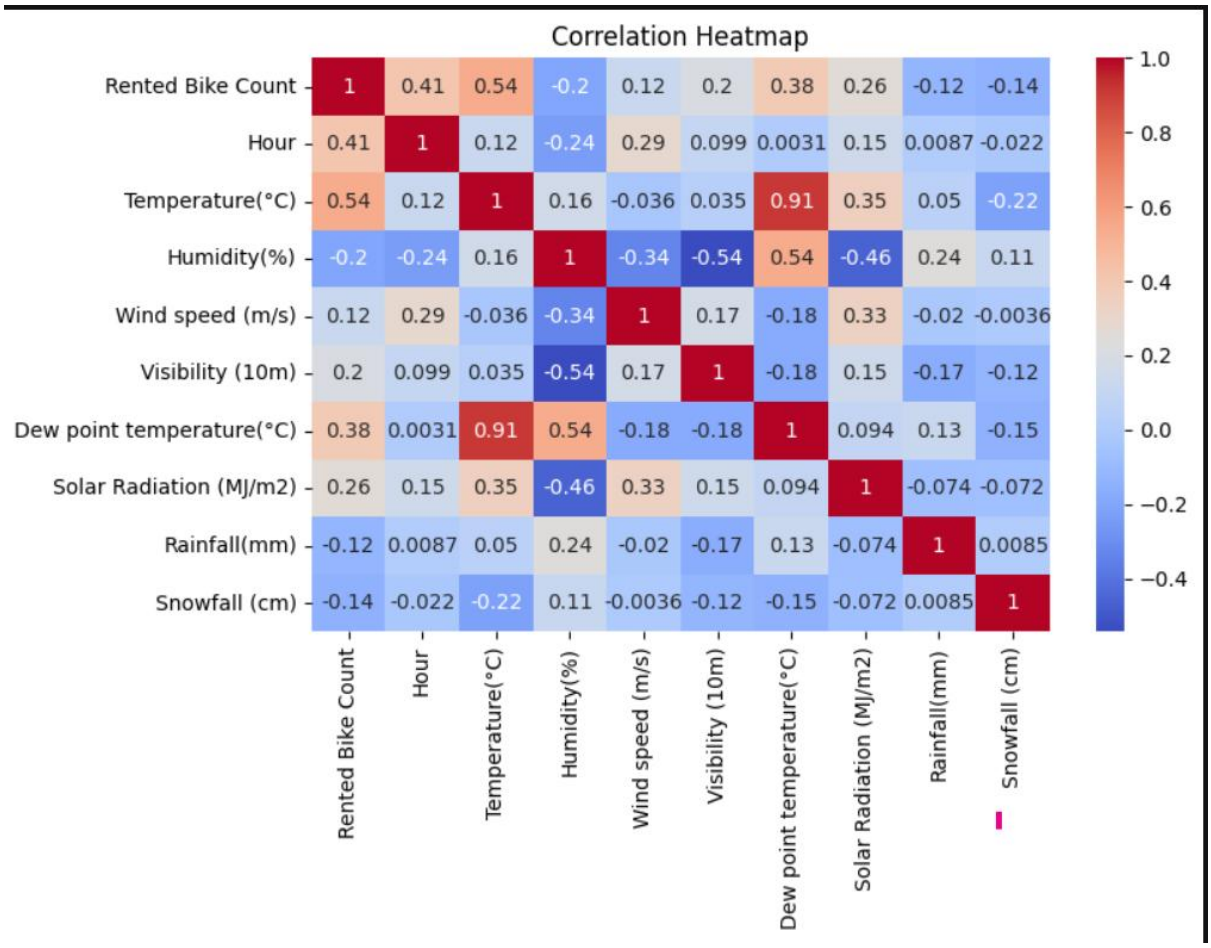
```
plt.figure(figsize=(6,3))
sns.barplot(data=Holiday_Bike_count,
            x='Holiday',
            y='Rented Bike Count')

plt.title('Average Bike Rentals vs Holiday')
plt.xlabel('Holiday')
plt.ylabel('Average Rented Bike Count')

plt.show()
```



```python
plt.figure(figsize=(6,3))
sns.barplot(data=Function_Bike_count,
            x='Functioning Day',
            y='Rented Bike Count')

plt.title('Average Bike Rentals vs Functioning Day')
plt.xlabel('Functioning Day')
plt.ylabel('Average Rented Bike Count')

plt.show()
```



Multivariate Analysis – Heatmap



Feature Engineering

```
#extract month from the date column

df['Month'] = df['Date'].dt.month
df['Month'].head()
```

✓ 0.0s

```
0    12
1    12
2    12
3    12
4    12
Name: Month, dtype: int32
```

```
#extract day of the week

df['Day_of_Week'] = df['Date'].dt.day_name()
df['Day_of_Week'].unique()
```

```
<StringArray>
['Friday', 'Saturday', 'Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday']
Length: 7, dtype: str
```

```
#Weekend Indicator

df['Weekend'] = df['Day_of_Week'].isin(['Saturday','Sunday']).astype(int)
df['Weekend'].unique() #0-weekdays 1-Weekends
```

```
array([0, 1])
```

Dropping date columns

```
#Drop the Date Column
df.drop('Date', axis=1, inplace=True)
```

Dataset after Feature Engineering

```
#Displaying the Df after Feature Engineering
df.head()
```

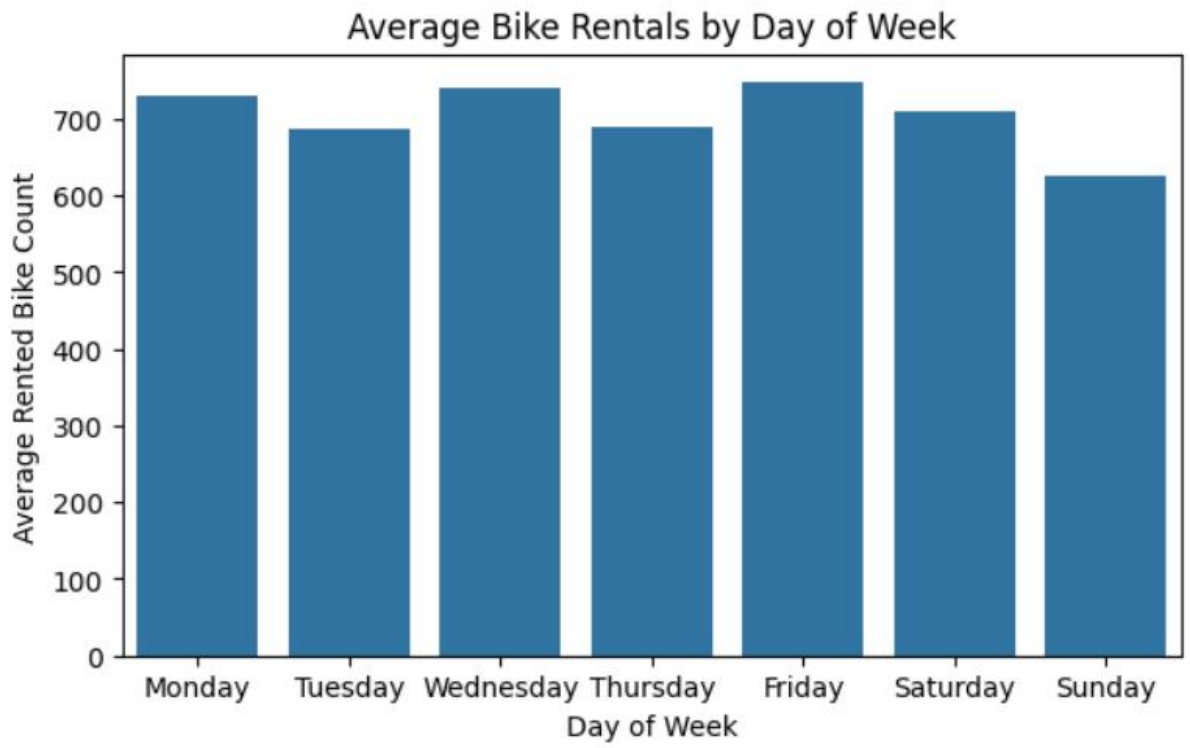
✓ 0.0s Python

	Rented Bike Count	Hour	Temperature(°C)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Dew point temperature(°C)	Solar Radiation (MJ/m2)	Rainfall(mm)	Snowfall (cm)	Seasons	Holiday	Functioning Day	Month	Day_of_Week	Weekend
0	254	0	-5.2	37	2.2	2000	-17.6	0.0	0.0	0.0	Winter	No Holiday	Yes	12	Friday	0
1	204	1	-5.5	38	0.8	2000	-17.6	0.0	0.0	0.0	Winter	No Holiday	Yes	12	Friday	0
2	173	2	-6.0	39	1.0	2000	-17.7	0.0	0.0	0.0	Winter	No Holiday	Yes	12	Friday	0
3	107	3	-6.2	40	0.9	2000	-17.6	0.0	0.0	0.0	Winter	No Holiday	Yes	12	Friday	0
4	78	4	-6.0	36	2.3	2000	-18.6	0.0	0.0	0.0	Winter	No Holiday	Yes	12	Friday	0

Ride Count on daily basis

```
sns.barplot(data=day_bike,x='Day_of_Week',y='Rented Bike Count')
plt.title('Average Bike Rentals by Day of Week')
plt.xlabel('Day of Week')
plt.ylabel('Average Rented Bike Count')
plt.show()
```

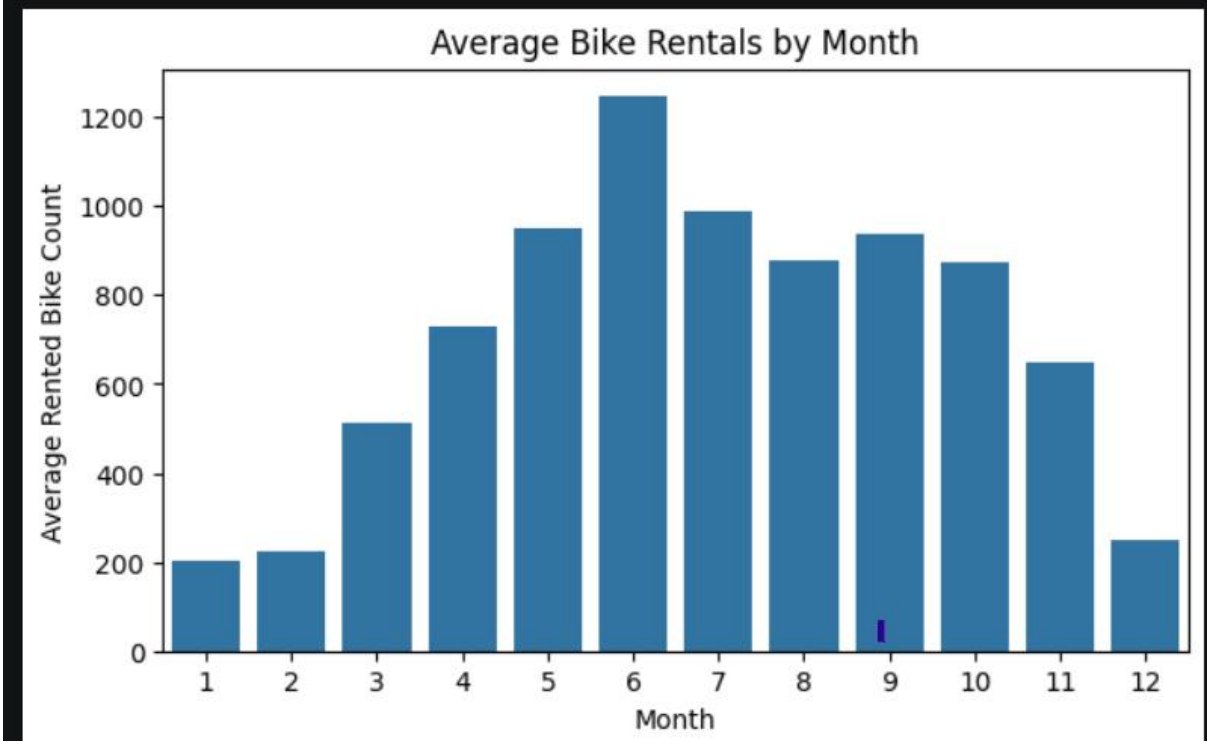
✓ 0.1s



Ride Count month wise

```
#barplot

plt.figure(figsize=(7,4))
sns.barplot(data=month_bike,x='Month',y='Rented Bike Count')
plt.title('Average Bike Rentals by Month')
plt.xlabel('Month')
plt.ylabel('Average Rented Bike Count')
plt.show()
```

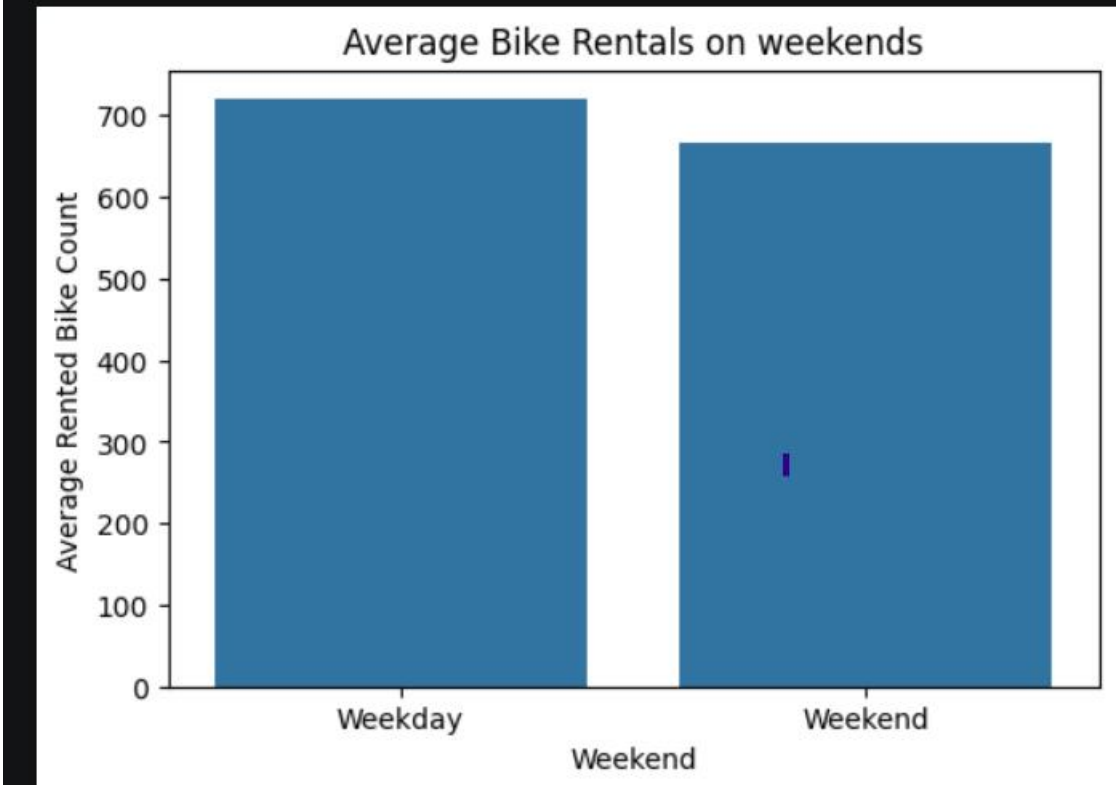
✓ 0.1s



Weekday vs weekend

```
plt.figure(figsize=(6,4))
sns.barplot(data=weekend_bike,x='Weekend',y='Rented Bike Count')
plt.xticks([0, 1], ['Weekday', 'Weekend'])
plt.title('Average Bike Rentals on weekends')
plt.xlabel('Weekend')
plt.ylabel('Average Rented Bike Count')
plt.show()
```

✓ 0.0s



Encoding Categorical Columns

Before

```
categorical_cols = df.select_dtypes(include='str')
categorical_cols.head()
```

✓ 0.0s

	Seasons	Holiday	Functioning Day
0	Winter	No Holiday	Yes
1	Winter	No Holiday	Yes
2	Winter	No Holiday	Yes
3	Winter	No Holiday	Yes
4	Winter	No Holiday	Yes

After

```
#importing required libraries
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df['Functioning Day'] = le.fit_transform(df['Functioning Day'])
print("Functioning Day:", dict(zip(le.classes_, le.transform(le.classes_))))

df['Holiday'] = le.fit_transform(df['Holiday'])
print("Holiday:", dict(zip(le.classes_, le.transform(le.classes_))))
```

✓ 0.3s

```
Functioning Day: {'No': np.int64(0), 'Yes': np.int64(1)}
Holiday: {'Holiday': np.int64(0), 'No Holiday': np.int64(1)}
```

```
#Performing One hot Encoding for Columns Seasons and Day_of_Week
df=pd.get_dummies(
    df,
    columns=['Seasons', 'Day_of_Week'],
    drop_first=True,
    dtype=int
)
```

✓ 0.0s

```
df.head()
```

✓ 0.0s Python

Weekend	Seasons_Spring	Seasons_Summer	Seasons_Winter	Day_of_Week_Monday	Day_of_Week_Saturday	Day_of_Week_Sunday	Day_of_Week_Thursday	Day_of_Week_Tuesday	Day_of_Week_Wednesd
0	0	0	1	0	0	0	0	0	
0	0	0	1	0	0	0	0	0	
0	0	0	1	0	0	0	0	0	
0	0	0	1	0	0	0	0	0	
0	0	0	1	0	0	0	0	0	

```
from sklearn.feature_selection import SelectKBest,f_regression

X=df.drop(labels=['Rented Bike Count','Dew point temperature(°C)'],axis=1)
y=df['Rented Bike Count']
```

✓ 0.1s

```
selector = SelectKBest(score_func=f_regression,k=12)
```

✓ 0.0s

```
X_new=selector.fit_transform(X,y)
X_new.shape
```

```
#view the selected columns
selected_cols= X.columns[selector.get_support()]
print(selected_cols)
```

✓ 0.0s

```
Index(['Hour', 'Temperature(°C)', 'Humidity(%)', 'Wind speed (m/s)',
       'Visibility (10m)', 'Solar Radiation (MJ/m2)', 'Rainfall(mm)',
       'Snowfall (cm)', 'Functioning Day', 'Month', 'Seasons_Summer',
       'Seasons_Winter'],
      dtype='str')
```

Feature Selection

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_new,y,test_size=0.2,random_state=42)
```

✓ 0.0s

```
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

✓ 0.0s

```
(7008, 12)
(7008,)
(1752, 12)
(1752,)
```

Splitting data

Feature Scaling

```
#import Standard Scaler

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

✓ 0.0s

Model Building and Tuning.

Before

```
#installing and imporing required libraries

#%pip install xgboost

import xgboost as xgb

xgb_model=xgb.XGBRegressor()
```

✓ 0.1s

```
xgb_model.fit(X_train,y_train)
```

✓ 0.1s

XGBRegressor

▸ Parameters

▸ Fitted attributes

```
xgb_y_pred = xgb_model.predict(X_test)
```

✓ 0.0s

```
print("Random Forest R2 score:", r2_score(y_test,rf_y_pred))
print("Random Forest Mean Absolute error:", mean_absolute_error(y_test,rf_y_pred))
print("Random Forest Mean Squared Error:", mean_squared_error(y_test,rf_y_pred))
print("Random Forest Root Mean Squared error:", root_mean_squared_error(y_test,rf_y_pred))
```

✓ 0.0s

```
Random Forest R2 score: 0.7324447937369396
Random Forest Mean Absolute error: 218.0940353777936
Random Forest Mean Squared Error: 111475.8700688091
Random Forest Root Mean Squared error: 333.8800234647307
```

After

```
xgb_model=xgb.XGBRegressor(
    objective='reg:squarederror',
    random_state=42
)
```

✓ 0.0s

```
xgb_params = {
    'n_estimators': [100, 200],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.05, 0.1],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}
```

✓ 0.0s

```
xgb_grid = GridSearchCV(
    estimator=xgb_model,
    param_grid=xgb_params,
    cv=5,
    scoring='r2',
    n_jobs=-1
)
```

✓ 0.0s

```
best_xgb = xgb_grid.best_estimator_

best_xgb_pred = best_xgb.predict(X_test)
```

✓ 0.0s

```
xgb_grid.fit(X_train, y_train)
```

✓ 32.8s

▸ GridSearchCV

▸ best_estimator_: XGBRegressor

▸ XGBRegressor

```
print("Best Parameters:", xgb_grid.best_params_)
print("Best CV Score:", xgb_grid.best_score_)
```

✓ 0.0s

```
Best Parameters: {'colsample_bytree': 0.8, 'learning_rate': 0.05, 'max_depth': 7, 'n_estimators': 200, 'subsample': 0.8}
Best CV Score: 0.8821609139442443
```

```
print("Performance after Tuning - XGBoost")
print("R2 Score:", r2_score(y_test, best_xgb_pred))
print("MAE:", mean_absolute_error(y_test, best_xgb_pred))
print("MSE:", mean_squared_error(y_test, best_xgb_pred))
print("RMSE:", root_mean_squared_error(y_test, best_xgb_pred))
```

✓ 0.0s

```
Performance after Tuning - XGBoost
R2 Score: 0.8749380111694336
MAE: 138.876708984375
MSE: 52106.6015625
RMSE: 228.26870727539062
```

Model Comparison

	Model	R^2 Score	MAE	MSE	RMSE	Remarks
0	Decision Tree (Tuned)	0.8015	169.07	82673.01	287.52	significantly improved after tuning
1	Random Forest (Tuned)	0.8676	141.51	55158.75	234.85	Significant performance improvement
2	XGBoost (Tuned)	0.8749	138.87	52106.60	228.26	Best tuned model

Best Model Summary

Metric	Value
Best Model	**XGBoost Regressor**
R^2 Score	**0.8749**
MAE	**138.87**
RMSE	**228.26**
Selected Because	Highest R^2 score and lowest prediction error among all evaluated models.

Deployment